

Peer-graded Assignment: Week 4: Non-parametric Models

Reviews 2 left to complete

Your fellow learner has submitted their assignment anonymously and your review will be anonymous to them. All names are still visible to course instructors.

Module4_v4

by Anonymous Learner
October 19, 2023

Like 17 Flag this submission

PROMPT

For Problem 1 of the Module 4 lab, you created a KNN classifier for handwritten digit recognition. Use your KNN class to perform KNN on the validation data with K = 3. Now you will check how well your classifier does. Create a **confusion matrix** in your Jupyter notebook. Feel free to use the Scikit-Learn confusion matrix (http://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html) function.

Upload a screenshot of the confusion matrix from your notebook here.

```
[[112  0  0  0  0  0  1  0  0  0]
 [  0 106  0  0  0  0  0  0  1  0]
 [  1  2 86  2  0  0  0  0  2  0]
 [  1  1  0 111  0  2  0  0  0  0]
 [  0  2  0  0 82  0  0  0  0  4]
 [  0  0  0  2  2 75  1  0  0  0]
 [  0  0  0  0  0  1 104  0  2  0]
 [  0  2  0  0  0  1  0 93  0  5]
 [  1  1  0  1  1  0  2  1 81  1]
 [  1  0  0  1  2  0  0  2  0 100]]
```

RUBRIC

Did the author upload a screenshot of a **confusion matrix** showing the performance of their KNN classifier on the 9 digits in the MNIST dataset?

See an example solution below:

```
confusion matrix:
[[112  0  0  0  0  0  1  0  0  0]
 [  0 106  0  0  0  0  0  0  1  0]
 [  1  2 86  2  0  0  0  0  2  0]
 [  1  1  0 111  0  2  0  0  0  0]
 [  0  2  0  0 82  0  0  0  0  4]
 [  0  0  0  2  2 75  1  0  0  0]
 [  0  0  0  0  0  1 104  0  2  0]
 [  0  2  0  0  0  1  0 93  0  5]
 [  1  1  0  1  1  0  2  1 81  1]
 [  1  0  0  1  2  0  0  2  0 100]]
```

2 pts

Learner's response looks reasonable for a confusion matrix showing the performance of their KNN classifier on the 9 digits in the MNIST dataset, e.g. they submitted a **10 x 10 confusion matrix**.

0 pts

Learner's response does not look reasonable for a confusion matrix showing the performance of their KNN classifier on the 9 digits in the MNIST dataset, e.g. they **didn't submit a 10 x 10 confusion matrix**.

PROMPT

Based on your confusion matrix, which digits seem to get confused with other digits the most?

Type your answer in the rich text field.

Digits 9 and 7 seem to be confused with other digits for having the most off-diagonal entries in the same row/column.

RUBRIC

Does the author correctly identify which digits are confused with other digits the most?

2 pts

Correctly identifies that 9 and 7 are confused with other digits the most.

See how the confusion matrix shows that these two digits were misclassified five times.

```
confusion matrix:
[[112  0  0  0  0  0  1  0  0  0]
 [  0 106  0  0  0  0  0  0  1  0]
 [  1  2 86  2  0  0  0  0  2  0]
 [  1  1  0 111  0  2  0  0  0  0]
 [  0  2  0  0 82  0  0  0  0  4]
 [  0  0  0  2  2 75  1  0  0  0]
 [  0  0  0  0  0  1 104  0  2  0]
 [  0  2  0  0  0  1  0 93  0  5]
 [  1  1  0  1  1  0  2  1 81  1]
 [  1  0  0  1  2  0  0  2  0 100]]
```

1 pt

Does not correctly identify that 9 and 7 are confused with other digits the most, but the digits identified are correct based on the confusion matrix that the learner uploaded.

0 pts

Learner did not answer the question **or** learner did not identify that 9 and 7 are confused with other digits the most **and** digits the learner identified were not correct based on the confusion matrix that the learner uploaded.

PROMPT

In your Module 4 Jupyter notebook, you created a plot of the accuracy of the KNN on the test set using the same set of axes for K=1, 2, ..., 20. You could go to K=30 if your implementation was efficient enough to allow it.

Please upload a **screenshot of your plot** here.

RUBRIC

Evaluate the learner's KNN accuracy plot.

Here is a sample solution plot:

2 pts

Plot has K values on the x-axis **and** test set accuracy on the y-axis.

2 pts

Plot **missing at least one of the following**: K values on x-axis **or** test set accuracy on the y-axis.

0 pts

No plot submitted.

PROMPT

Based on the plot of the KNN accuracy on the test set, which value of K results in highest accuracy?

Type your answer in the rich text field.

The accuracy peaks at K = 3 according to the figure.

RUBRIC

From the solution KNN accuracy plot below, we can see that a K value of 3 neighbors results in the highest accuracy.

4 pts

Learner correctly identifies that 3 neighbors results in the highest accuracy.

2 pts

Learner does not identify a K value of 3 but identifies the highest accuracy K value on the plot that the learner submitted for Prompt 3.

0 pts

No answer attempted **or** learner does not identify highest accuracy K value on the plot that the learner submitted.

PROMPT

In Problem 2 about Decision Trees, the directions for Part C1 are to:

1. Modify **max_depth**:

- Create a model with a shallow **max_depth** of 2. Build the model on the training set with the **build_dt** function.

- Report **precision/recall** on the test set.

- Report **depth of the tree**.

Upload a screenshot of your code for this section.

```
# Part C1: Generate the model output for max_depth
def build_dt(X_train, y_train, max_depth=2):
    """Build a decision tree model with a given max_depth.
    Parameters:
    X_train (array-like): Training features.
    y_train (array-like): Training labels.
    max_depth (int): Maximum depth of the tree.
    Returns:
    A fitted DecisionTreeClassifier model.
    """
    from sklearn.tree import DecisionTreeClassifier
    model = DecisionTreeClassifier(max_depth=max_depth)
    model.fit(X_train, y_train)
    return model

# Generate model output
model = build_dt(X_train, y_train, max_depth=2)
y_test_hat = model.predict(X_test)
```

RUBRIC

Here is a **solution**:

```
1 dt = build_dt(X_train, y_train, max_depth= 2)
2
3
4 y_test_hat = dt.predict(X_test)
5
6
7 prediction = calculate_precision(y_test, y_test_hat)
8
9
10 recall = calculate_recall(y_test, y_test_hat)
11
12 print(f"precision: {:.2f} Recall: {:.2f} Tree Depth: {}".format(prediction, recall, dt.get_depth()))
```

Evaluate the learner's submission.

5 pts

Includes all of the following: Learner calls build_dt function with max_depth=2 **and** model built on the training set **and** calculates precision on the test set **and** calculates recall on the test set **and** reports depth of the tree.

4 pts

Missing one of the following: Learner calls build_dt function with max_depth=2 **or** model built on the training set **or** calculates precision on the test set **or** calculates recall on the test set **or** reports depth of the tree.

3 pts

Missing two of the following: Learner calls build_dt function with max_depth=2 **or** model built on the training set **or** calculates precision on the test set **or** calculates recall on the test set **or** reports depth of the tree.

2 pts

Missing three of the following: Learner calls build_dt function with max_depth=2 **or** model built on the training set **or** calculates precision on the test set **or** calculates recall on the test set **or** reports depth of the tree.

1 pt

Missing four of the following: Learner calls build_dt function with max_depth=2 **or** model built on the training set **or** calculates precision on the test set **or** calculates recall on the test set **or** reports depth of the tree.

0 pts

Missing all of the following: Learner calls build_dt function with max_depth=2 **and** model built on the training set **and** calculates precision on the test set **and** calculates recall on the test set **and** reports depth of the tree.

PROMPT

In Problem 2 about Decision Trees, the directions for Part C1 are to:

2. Modifying **max_leaf_nodes**:

- Create a model with a shallow **max_leaf_nodes** of 4. Build the model on the training set.

Upload a screenshot of your code for this section.

```
# Part C2: Generate the model output for max_leaf_nodes
def build_dt(X_train, y_train, max_leaf_nodes=4):
    """Build a decision tree model with a given max_leaf_nodes.
    Parameters:
    X_train (array-like): Training features.
    y_train (array-like): Training labels.
    max_leaf_nodes (int): Maximum number of leaf nodes in the tree.
    Returns:
    A fitted DecisionTreeClassifier model.
    """
    from sklearn.tree import DecisionTreeClassifier
    model = DecisionTreeClassifier(max_leaf_nodes=max_leaf_nodes)
    model.fit(X_train, y_train)
    return model

# Generate model output
model = build_dt(X_train, y_train, max_leaf_nodes=4)
y_test_hat = model.predict(X_test)
```

RUBRIC

Here is a solution:

```
11
```

Evaluate the learner's solution.

5 pts

Includes all of the following: Learner calls build_dt function with max_leaf_nodes = 4 **and** model built on the training set

<https://www.coursera.org/learn/introduction-to-machine-learning/supervised-learning/peer-ids2c/week-4-non-parametric-models/review-test>

1/1